

Entregable 7

1. Mecanismo de Autenticación Elegido al Abrir SSE

Para este entorno de ejecución (Script de cliente en Python), el mecanismo elegido es inyectar el token en el **Header HTTP** (**Authorization: Bearer <token>**).

Justificación: A diferencia de la API `EventSource` en navegadores (que restringe la inyección de headers personalizados y nos obligaría a usar `Cookies HttpOnly`), las librerías HTTP de Python (como `requests`) permiten enviar headers libremente en peticiones de streaming. Esto evita exponer el token en la URL (como ocurriría si usáramos `query parameters`) y mantiene la arquitectura limpia y alineada a los estándares REST.

2. Diagrama de Flujo: Reconexión Autenticada y Manejo de Expiración

A continuación se describe el flujo lógico del "Doble Ciclo" que gobierna la conexión:

1. **(Ciclo Principal) Pre-validación:** Antes de intentar conectar, el cliente consulta al `TokenManager` (`is_expiring_soon()`).
 - Si expira pronto: Ejecuta `refresh_access_token()`.
2. **Conexión Inicial:** Se solicita el token actual y se inyecta en el header. Se abre la conexión HTTP en modo *stream*.
3. **Validación de Conexión:**
 - Si el servidor responde `HTTP 401`: El token ya expiró. El cliente fuerza un refresh y vuelve al Paso 1.
 - Si hay error de red (`Timeout`): El cliente espera 5 segundos (`backoff`) y vuelve al Paso 2 (sin borrar el token, por resiliencia).
 - Si es `HTTP 200`: Se inicia la lectura de eventos (Ciclo Interno).
4. **(Ciclo Interno) Escucha Activa:** El cliente lee línea por línea.
 - *Evento de negocio:* Se procesa el evento de EcoMarket.
 - *Evento de expiración de token:* Si el servidor envía amablemente un evento tipo `event: auth_error` cerrando el stream, el cliente rompe el ciclo interno de lectura y el ciclo principal retoma el control, forzando un refresh y nueva conexión (Vuelta al Paso 1).

3. Pseudocódigo de la Integración

Python

import time

class ClienteSSEMultiplex:

```
def __init__(self, token_manager, url_sse):
    self.token_manager = token_manager
    self.url_sse = url_sse
```

```
def conectar_y_escuchar(self):
```

```
    # --- CICLO EXTERNO: Control de conexión y re-autenticación ---
```

```
    while True:
```

```
        # 1. Proactividad: Evitar conectar con un token a punto de morir
```

```
        if self.token_manager.is_expiring_soon():
```

```
            print("Token próximo a expirar. Renovando proactivamente...")
```

```
            if not self.token_manager.refresh_access_token():
```

```
                print("Error crítico: No se pudo renovar el token. Sesión terminada.")
```

```
                break # Salida definitiva (requiere re-login del usuario)
```

```
        # 2. Obtener la llave maestra actual (Mecanismo elegido: Header)
```

```
        token_actual = self.token_manager.get_access_token()
```

```
        headers = {"Authorization": f"Bearer {token_actual}"}
```

```
        try:
```

```
            # 3. Intentar conexión al stream SSE
```

```
            respuesta = requests.get(self.url_sse, headers=headers, stream=True)
```

```
            # Si el servidor rechaza el token en la puerta (ej. expiración no detectada por tiempo
desfasado)
```

```
            if respuesta.status_code == 401:
```

```
                print("Rechazo 401. Forzando renovación de token...")
```

```
                self.token_manager.refresh_access_token()
```

```
                continue # Reintentar el ciclo con el nuevo token
```

```
            respuesta.raise_for_status()
```

```
            print("Conexión SSE establecida con éxito.")
```

```
        # --- CICLO INTERNO: Lectura del stream ---
```

```
        for linea in respuesta.iter_lines():
```

```
            if linea:
```

```
                evento = linea.decode('utf-8')
```

```
        # 4. Reactividad: El servidor decide cerrar la conexión por expiración
```

```
        if "event: token_expired" in evento:
```

```
            print("El servidor cerró la sesión por tiempo. Reconectando...")
```

```
            break # Rompe este 'for', vuelve al 'while' para reconectar
```

```
procesar_evento(evento)
```

```
except request_error as e:
```

```
    # 5. Resiliencia: Fallo temporal de red
```

```
    print(f"Error de red: {e}. Reintentando en 5 segundos...")
```

```
    time.sleep(5)
```

```
    # El token no se borra. Se reintentará la conexión.
```